

FPGA-Based Smart Hospital Emergency Response Dispatcher: VHDL Design, Verification, FPGA Demonstration, and Altium PCB Realization

Hardware Engineering Laboratory – Team C2

Bertrand Mungu Cho, Daniel Chidi Chimezie, Riway Ghimire, Stephen Uzochi Obuh

Hochschule Hamm-Lippstadt, Germany

Summer Semester 2026

Abstract—This report presents the complete development of an FPGA-based Smart Hospital Emergency Response Dispatcher implemented in VHDL and demonstrated on a Digilent Nexys A7 board. The work progressed from fundamental combinational and sequential laboratory exercises to a modular safety-oriented dispatcher comprising patient acquisition, triage priority encoding, resource allocation, finite-state scheduling, patient parking and selective release, fault supervision, visual status indication, seven-segment output, and UART transmission. The repository contains ten synthesizable VHDL modules and ten dedicated testbenches with 114 assertion statements. A compiled bitstream and board constraint file provide implementation evidence, while the successful laboratory demonstration confirmed the intended interactive workflow. In parallel, a seven-sheet Altium design, two-layer PCB layout, 3-D board model, 121-row bill of materials, and complete Gerber/drill package were produced. The result demonstrates an end-to-end hardware engineering workflow from Boolean logic and RTL verification through FPGA deployment and manufacture-oriented PCB documentation.

Index Terms—FPGA, VHDL, Vivado, Nexys A7, Artix-7, finite-state machine, resource scheduling, Altium Designer, printed circuit board, hardware verification.

I. INTRODUCTION

Field-programmable gate arrays (FPGAs) support deterministic parallel execution, explicit timing control, and direct hardware interfacing, making them suitable for teaching real-time control and safety-related digital design. The Team C2 laboratory work used these properties to develop a progressively more complex hardware system. Early exercises established correct use of VHDL for Boolean gates, adders, subtractors, binary-coded-decimal arithmetic, seven-segment decoding, clock division, and decimal counters. These foundations were then integrated into a final project that models the dispatch process of a hospital emergency department.

The target platform was the Digilent Nexys A7, built around an AMD Artix-7 FPGA and providing a 100-MHz clock, switches, push buttons, LEDs, seven-segment displays, and expansion connectors [1], [2]. Vivado was used for RTL synthesis, implementation, constraint handling, bitstream generation, and simulation in accordance with the standard FPGA flow described in AMD documentation [3]–[6]. The design sources use the IEEE VHDL and multivalued logic standards [7], [8].

TABLE I
LABORATORY PROGRESSION

Lab	Main work	Relevance to final project
01	XOR, half/full adders and subtractors	Combinational design, entities, architectures, exhaustive simulation
02	Ripple-carry adder/subtractor, BCD adder, BCD-to-seven-segment decoder	Hierarchical RTL, arithmetic datapaths, display encoding
04	Altium PCB workflow	Schematic capture, footprints, placement, routing, manufacturing data
05	FPGA bitstreams and XDC constraints	Mapping RTL ports to physical Nexys A7 resources
07	Clock divider and one-/two-digit counters	Sequential logic, clocked processes, display multiplexing
Final	Smart Hospital Dispatcher	Integrated control, scheduling, safety, communication, PCB realization

The principal achievements are: (i) a modular ten-component VHDL architecture, (ii) assertion-based unit and integration testbenches, (iii) a board-ready top-level design and bitstream, (iv) a successful interactive laboratory demonstration, and (v) a complete Altium PCB design package extending the FPGA concept toward a dedicated interface board.

II. LABORATORY DEVELOPMENT PATH

The project followed an incremental development strategy. Table I summarizes the progression and its contribution to the final system.

This progression reduced integration risk: individual concepts were verified before they were reused in the larger design. Such staged refinement is consistent with standard FPGA prototyping practice [9], [10].

III. SYSTEM ARCHITECTURE AND OPERATION

The dispatcher receives patient data from switches and control events from push buttons. Severity and patient type are encoded into a priority value; the scheduler then checks and allocates rooms, doctors, and equipment. A safety supervisor observes invalid or unsafe operating conditions, while visual and serial interfaces expose the system state.

The implemented top level structurally instantiates ten modules:

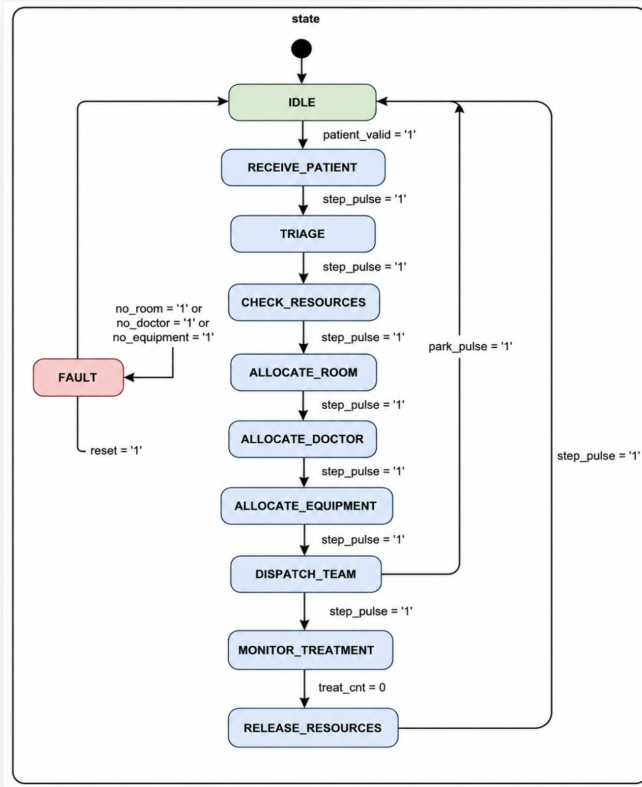


Fig. 1. Project finite-state control concept used to structure the dispatcher workflow.

- 1) `patient_generator`: synchronizes the new-patient button, detects a rising edge, and captures ID, type, and severity.
- 2) `priority_encoder`: converts triage inputs into a scheduling priority.
- 3) `resource_manager`: tracks four rooms, four doctors, and four equipment units; it supplies free indices and no-resource flags.
- 4) `scheduler_fsm`: sequences reception, triage, resource checking, allocation, dispatch, monitoring, parking, release, and fault handling.
- 5) `patient_table`: stores up to four parked assignments and enables selective release by slot.
- 6) `safety_supervisor`: detects resource and consistency faults and latches an alarm.
- 7) `led_controller`: maps occupancy and safety status to 16 LEDs and blinks the alarm indication.
- 8) `seven_segment_driver`: multiplexes numerical/state information onto the board displays.
- 9) `uart_controller`: transmits 8N1 serial data at 115200 baud using a divisor of 868 for the 100-MHz clock.
- 10) `top_level`: integrates the complete system, merges release sources, gates button functions, and sequences UART status bytes.

The design uses two-stage input synchronization and edge detection where asynchronous push buttons enter the clock domain. This is an important practical measure because unsynchronized inputs can violate setup or hold time and

TABLE II
MAJOR DEMONSTRATED OPERATING SCENARIOS

Scenario	Expected observable behavior
Normal dispatch	Patient captured, triaged, resources allocated, treatment indicated
Critical patient	Critical flag asserted and priority reflected in the dispatch path
Emergency override	Scheduler follows the emergency control branch
Parking	Current resource indices stored in the next free patient-table slot
Selective release	Chosen slot emits release pulses and clears its occupied bit
Resource exhaustion	No-resource flags prevent unsafe allocation and invoke fault handling
Reset	State, occupancy, parked slots, display status, and latched alarm return to defaults

lead to metastable behavior. A more complete product version would additionally debounce all mechanical controls.

IV. SCHEDULING, RESOURCE MANAGEMENT, AND SAFETY

The scheduler is the central control element. Patient severity and type are latched at admission, after which availability signals determine whether allocation can proceed. Room, doctor, and equipment allocation are separated into explicit actions so that each resource transition is observable and testable. Treatment can either proceed to normal release or be parked, preserving the allocated indices in one of four patient-table slots. In release mode, a selected slot produces one-cycle release strobes with the corresponding stored indices.

The design therefore supports more than a simple single-patient demonstration. It can retain multiple active allocations, display aggregate occupancy, selectively free resources, and exercise no-resource behavior. Emergency override and fault paths give the state machine explicit responses to exceptional conditions rather than allowing implicit or undefined behavior. The alarm latch and blinking LED make a fault persistent and visible until reset.

V. VERIFICATION AND FPGA IMPLEMENTATION

Verification was organized at module level and at the integrated top level. The repository contains ten testbenches corresponding to the ten synthesizable design units. Across these files, 114 VHDL assertion statements compare observed behavior with expected values and report failures. This provides repeatable evidence for edge detection, priority mapping, allocation/release behavior, state transitions, display encoding, LED mapping, UART framing, alarm behavior, parking, and top-level interaction.

The presence of a top-level bitstream and the Nexys A7 master constraint file confirms that the design advanced beyond behavioral simulation into implementation. The constraint file maps the 100-MHz board clock, switches, buttons, LEDs, seven-segment interface, and UART connection. The generated bitstream was loaded on the laboratory board, and the final demonstration worked as intended. The photographed setup in Fig. 2 provides physical evidence of the hardware validation.



Fig. 2. Laboratory demonstration setup used for physical validation of the FPGA implementation.

A strict quantitative timing or utilization claim is intentionally not made because exported Vivado timing and utilization reports were not included in the archive. For final design sign-off, the next verification package should include post-implementation timing summary, clock-domain and design-rule reports, resource utilization, power estimation, and waveform exports. The current evidence nevertheless supports functional implementation and successful board demonstration.

VI. ALTIVM SCHEMATIC AND PCB DESIGN

The Altium work extends the FPGA implementation into a manufacture-oriented custom interface board. The schematic is divided into seven sheets covering power, FPGA/core interface, clock/configuration/JTAG, inputs, LED outputs, seven-segment displays, and buzzer/UART. This hierarchical partitioning improves reviewability and limits schematic complexity.

The board is specified as a two-layer mixed through-hole and surface-mount design. It accepts 5 V input and includes 3.3-V, 1.8-V, and 1.0-V regulated rails with local decoupling. A custom 56-pin FPGA interface connects the external Nexys A7. User interaction is provided by eight slide switches, six push buttons, 16 LEDs, four dual-digit seven-segment displays, a buzzer, UART, and JTAG headers.

The bill of materials contains 120 populated data rows plus its header. The manufacturing package includes top and bottom copper, solder mask, silkscreen/legend, paste layers, board profile, plated-through-hole drill, and non-plated-through-hole drill outputs. This is a substantial achievement because it demonstrates the complete transition from schematic intent to fabrication data rather than stopping at a visual PCB layout. Altium's integrated flow similarly emphasizes consistency

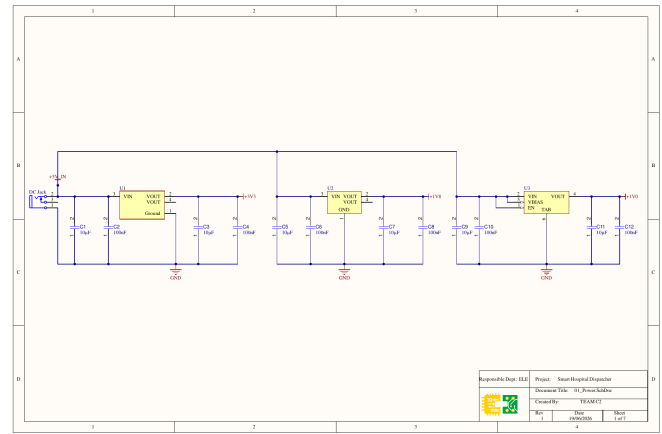


Fig. 3. Altium power and interface schematic sheet from the seven-sheet design set.

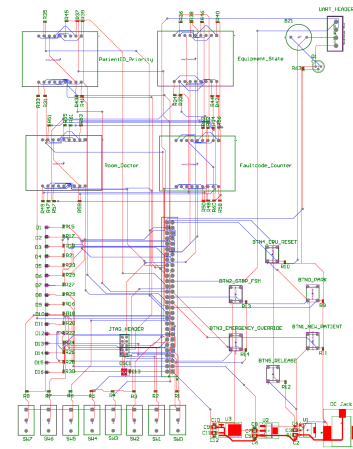


Fig. 4. Composite two-layer PCB layout exported from Altium Designer.

between schematic capture, PCB implementation, and output generation [11].

VII. RESULTS AND ENGINEERING ASSESSMENT

The final result satisfies the principal educational objectives of the Hardware Engineering Laboratory. The team demonstrated competence across four linked domains: digital logic fundamentals, modular RTL development, verification and FPGA deployment, and PCB engineering. The strongest evidence is the consistency between the VHDL architecture, dedicated testbenches, board constraints, generated bitstream, successful physical demonstration, Altium schematic set, PCB layout, BOM, and Gerber outputs.

The design also contains several features that exceed a minimal classroom dispatcher: asynchronous-input synchronization, occupancy vectors, indexed release, four parked patient slots, alarm latching, blinking safety indication, display multiplexing, and UART reporting. The architecture is understandable because responsibilities are separated by module rather than concentrated in a monolithic process.

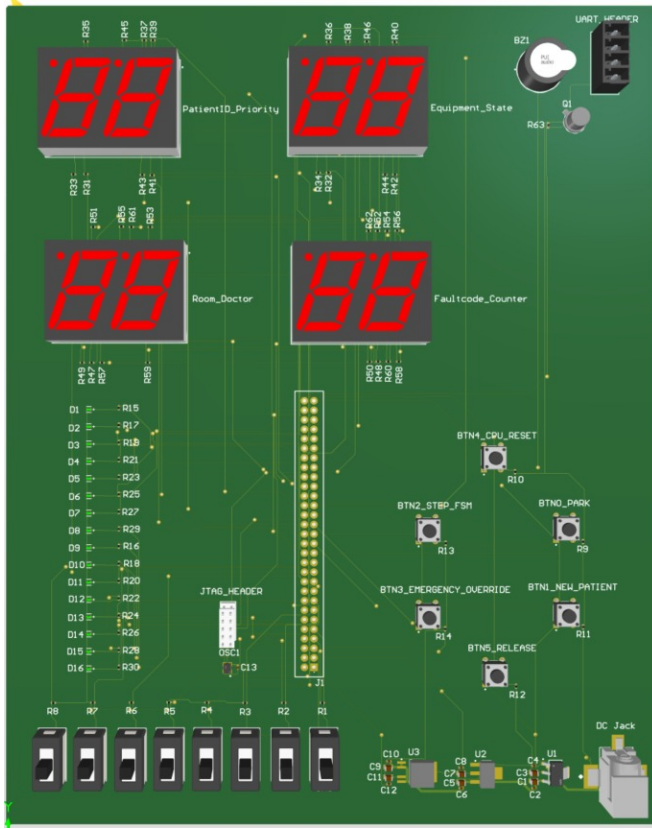


Fig. 5. Three-dimensional rendering of the Smart Hospital Dispatcher interface PCB.

There are nevertheless clear opportunities for improvement. Mechanical buttons should be debounced; resource requests could be queued instead of processing only the current patient transaction; severity ordering should be formally specified and covered exhaustively; UART messages could use framed packets with checksums; and safety requirements could be expressed as formal properties. On the PCB side, manufacturing release should be preceded by electrical-rule checks, design-rule checks, review of regulator stability and thermal behavior, connector pin-by-pin validation against the Nexys A7 electrical limits, and prototype bring-up tests. Exported DRC/ERC reports and a tested assembled PCB would convert the manufacture-ready design evidence into full hardware validation.

VIII. CONCLUSION

Team C2 completed an end-to-end hardware engineering project centered on an FPGA-based Smart Hospital Emergency Response Dispatcher. The work progressed from basic VHDL components to a ten-module integrated design with ten testbenches and 114 assertions, a constrained Nexys A7 implementation, a generated bitstream, and a successful laboratory demonstration. In parallel, the team created a seven-sheet Altium schematic, two-layer PCB layout, 3-D model, detailed BOM, and complete Gerber/drill package. The project therefore demonstrates not only functional digital design, but also a coherent engineering process linking specification,

RTL architecture, verification, implementation, user interaction, safety handling, and fabrication documentation. The remaining work is primarily quantitative sign-off and physical PCB prototyping rather than fundamental redesign.

ACKNOWLEDGMENT

The authors thank the Hardware Engineering Laboratory teaching staff at Hochschule Hamm-Lippstadt for the laboratory platform, technical guidance, and opportunity to demonstrate the system on physical FPGA hardware.

REFERENCES

- [1] Digilent, "Nexys A7 FPGA Board Reference Manual," 2024.
- [2] AMD, "Artix 7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181, ver. 1.27.1, Jul. 2024.
- [3] AMD, "Vivado Design Suite User Guide: Logic Simulation," UG900, 2024.
- [4] AMD, "Vivado Design Suite User Guide: Synthesis," UG901, 2025.
- [5] AMD, "Vivado Design Suite User Guide: Using Constraints," UG903, 2024.
- [6] AMD, "Vivado Design Suite User Guide: Implementation," UG904, 2024.
- [7] IEEE, "IEEE Standard for VHDL Language Reference Manual," IEEE Std 1076-2019, 2019.
- [8] IEEE, "IEEE Standard Multivalued Logic System for VHDL Model Interoperability," IEEE Std 1164-1993, 1993.
- [9] P. P. Chu, *FPGA Prototyping by VHDL Examples*, 2nd ed. Hoboken, NJ, USA: Wiley, 2017.
- [10] M. M. Mano and M. D. Ciletti, *Digital Design*, 6th ed. Pearson, 2018.
- [11] Altium, "Altium Designer Documentation: Schematic Capture, PCB Layout, and Manufacturing Outputs," 2026.
- [12] AMD, "UltraFast Design Methodology Guide for FPGAs and SoCs," UG949, 2024.